Addresses, Byte Order and the Structure of a Client and a Server

Lecture 08

Computer Networks and Distributed Programming

Where we are: the numbers you typed in

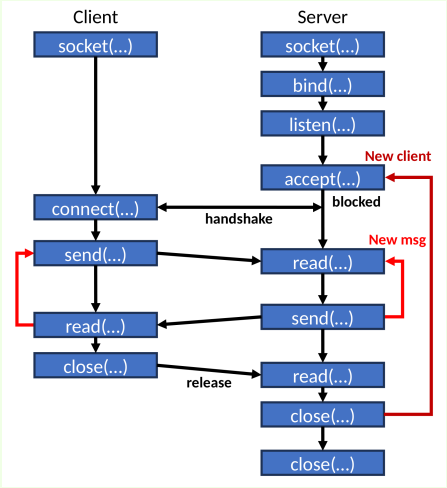
Last time both programs worked, and three things in them were given to you without a word of explanation.

- `5000` — who is allowed to choose a port, and which ones are free to choose?
- `htons` — why does a number need converting before it goes into a struct?
- `127.0.0.1` — and how do you write a client for a machine you only know by name?

Today those three, and then a client that leaves this room: an HTTP request written by hand against a real web server.

The shape of the two programs

- The calls are the ones you already wrote; what the picture adds are the two loops.
- The inner one is a conversation: read and reply until the client has nothing more to say.
- The outer one is the server's life: after `close`, back to `accept` for the next client.
- Everything still missing is in the arguments of those calls — the addresses.



An address is an IP address and a port

Definition

A communication endpoint is identified by a 32-bit IP address, which selects the host, and a 16-bit port, which selects one program on it.

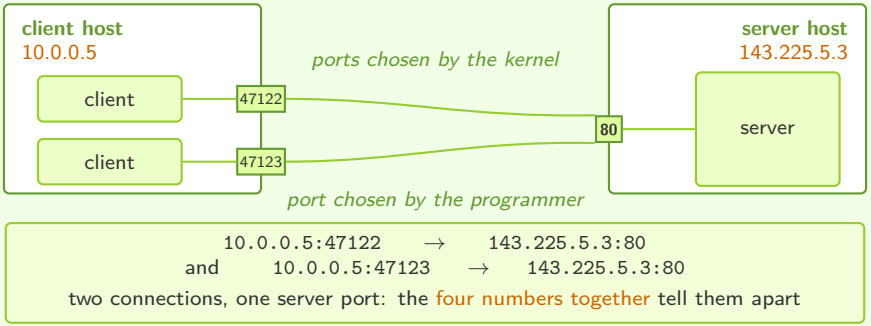
- The address gets the bytes to the machine; without the port the kernel would not know whom to hand them to.
- That is why one host can run a web server, a mail server and your chat server at the same time.
- Sixteen bits means 0 to 65535, and the number is written into every packet header.
- Both numbers together are what `bind` and `connect` take: nothing else is negotiated.

Which port you are allowed to use

Range	Name	Who uses it
0–1023	well known	servers of standard services, root only
1024–49151	registered	assigned on request to applications
49152–65535	dynamic	handed out by the kernel, never assigned

- The only boundary that matters to you is 1024: below it your server will not start unless you are root.
- Linux hands out its ephemeral ports from 32768 to 60999, which is not the dynamic range above and is a tunable.
- Well known ones you already met: 22 `ssh`, 25 `smtp`, 53 `dns`, 80 `http`, 443 `https`.
- Pick something above 1024 for your own servers, and expect address already in use if a neighbour picked the same.

A connection is four numbers, not two



The client never picks its own port and never says its own address: the kernel fills both in. The server learns them from the connection, which is why `accept` can tell you who called.

Seeing the four numbers

```
$ ss -ltn                       listening TCP sockets, numeric
State   Recv-Q  Send-Q   Local Address:Port    Peer Address:Port
LISTEN  0       16       0.0.0.0:5000          0.0.0.0:*

$ ss -tn                        two clients, same server port
ESTAB   0       0        127.0.0.1:5000        127.0.0.1:47122
ESTAB   0       0        127.0.0.1:5000        127.0.0.1:47123
```

- `0.0.0.0:5000` is the listening socket: no peer, because it never carries data.
- The two `ESTAB` lines differ only in the last number, and that is enough.
- `netstat -tan` prints the same thing and is the older command you will find in books.

The same number, two layouts in memory

The integer 513 occupies four bytes, and the two families of processors write them in opposite orders.

Address	Big endian	Little endian
0	00000000	00000001
1	00000000	00000010
2	00000010	00000000
3	00000001	00000000

- Big endian puts the most significant byte first; little endian the least. Neither is wrong.
- Your machine is almost certainly little endian; the protocols of the Internet are big endian.
- So the conversion is not optional, and it is not something the compiler can do for you.

Network byte order, and four functions

```c
#include <arpa/inet.h>

uint16_t htons(uint16_t v);      /* host    -> network, 16 bit: a port    */
uint32_t htonl(uint32_t v);      /* host    -> network, 32 bit: an IPv4   */
uint16_t ntohs(uint16_t v);      /* network -> host,    16 bit            */
uint32_t ntohl(uint32_t v);      /* network -> host,    32 bit            */
```

- Read the names: `h` host, `n` network, `s` short (16 bit), `l` long (32 bit).
- On a big-endian machine they do nothing, and that is the point: the same source compiles and runs everywhere.
- Convert when the value enters the struct and convert back when it comes out. Never twice.

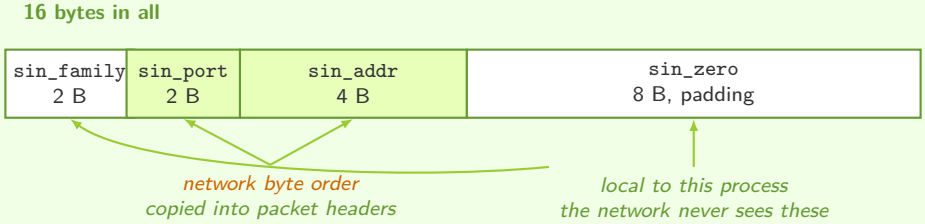
What a missing htons costs

The server binds `5000` without `htons`. The client connects to `htons(5000)`. What happens?

They do not meet. `5000` is `0x1388`; stored the other way round it reads `0x8813`, that is 34835. The server is listening on a port nobody asked for.

- The client gets connection refused and the mistake looks like a network problem, which it is not.
- Worse is the symmetric case: both sides forget, both are wrong in the same way, and everything works.
- It will keep working until one side runs on a big-endian machine, or until you check with `ss`.

Which fields actually travel



This is the whole rule: convert what is copied into a header, leave alone what the kernel reads locally. `sin_family` is never converted, and `sin_zero` exists only to reach sixteen bytes.

Your data needs it too

```c
uint32_t v = htonl(12345);              /* sender: convert, then send  */
write(fd, &v, sizeof v);

uint32_t net;                           /* receiver: all four bytes    */
size_t got = 0;
while (got < sizeof net) {
    ssize_t n = read(fd, (char *) &net + got, sizeof net - got);
    if (n <= 0) { /* peer gone, or error */ break; }
    got += (size_t) n;
}
uint32_t back = ntohl(net);
```

Byte order is not a property of addresses: it is a property of every number you put on the wire. The loop is there because four bytes may arrive in two pieces.

Make the client send two numbers and the server reply with their sum, in a loop until you interrupt it.

From a dotted string to four bytes

```c
struct sockaddr_in a;
char txt[INET_ADDRSTRLEN];

if (inet_pton(AF_INET, "143.225.5.3", &a.sin_addr) != 1) {   /* 1 = ok */
    fprintf(stderr, "not a valid address\n"); exit(EXIT_FAILURE);
}
inet_ntop(AF_INET, &a.sin_addr, txt, sizeof txt);            /* and back */
```

- `inet_pton` writes straight into `sin_addr`, already in network order: no `htonl` afterwards.
- It returns 1 on success, 0 if the string is malformed, -1 if the family is not supported.
- `inet_ntop` goes the other way and needs a buffer from you. The older `inet_addr` and `inet_ntoa` are best avoided.

The server's address, the client's address

```c
struct sockaddr_in me;                          /* server: its own address */
memset(&me, 0, sizeof me);
me.sin_family      = AF_INET;
me.sin_port        = htons(5000);               /* chosen by you           */
me.sin_addr.s_addr = htonl(INADDR_ANY);         /* any local interface     */
bind(fd, (struct sockaddr *) &me, sizeof me);

struct sockaddr_in srv;                         /* client: the other one   */
memset(&srv, 0, sizeof srv);
srv.sin_family = AF_INET;
srv.sin_port   = htons(5000);                   /* must match the server   */
inet_pton(AF_INET, "143.225.5.3", &srv.sin_addr);
connect(fd, (struct sockaddr *) &srv, sizeof srv);   /* no bind */
```

Same struct, opposite meaning: `bind` says where I am, `connect` says where you are.

INADDR_ANY, or one interface?

Your server works with `nc` on your own machine and is unreachable from the laptop next to you. Why?

Because it was bound to 127.0.0.1. That address exists on every machine and leaves none of them: the packets from outside arrive on a different interface, and nothing is listening there.

- A host has several addresses — loopback, cable, wireless — and `bind` attaches you to one of them.
- `INADDR_ANY`, printed as `0.0.0.0`, means every interface this machine has, present and future.
- It is the normal choice for a server, and the deliberate exception is a service that must not be reachable from outside.

Restarting the server: socket options

```
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);

struct timeval tv = { .tv_sec = 5, .tv_usec = 0 };
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
```

- Options live in levels: `SOL_SOCKET` here, `IPPROTO_TCP` and `IPPROTO_IP` lower down.
- `SO_REUSEADDR` lets `bind` take a port that the previous run has closed but the kernel is still holding.
- That wait is the orderly release of the old connection, and we will see what it protects when we open TCP up.
- `SO_RCVTIMEO` makes a blocking `read` give up after five seconds, returning -1 with `errno` set.

The same code over IPv6

```
struct sockaddr_in6 me6;                    /* 16-byte address inside  */
memset(&me6, 0, sizeof me6);
me6.sin6_family = AF_INET6;
me6.sin6_port   = htons(5000);              /* htons, exactly as before */
me6.sin6_addr   = in6addr_any;              /* the wildcard, as ::      */
bind(fd6, (struct sockaddr *) &me6, sizeof me6);
```

- Different struct, different family, same six calls: the API was designed so that this is all that changes.
- The socket must be created with `AF_INET6`, and the cast to `struct sockaddr *` is the reason that generic type exists.
- Writing both versions by hand is tedious and avoidable — which is what the next few frames are about.

A socket takes an address, not a name

- Everything so far needed numbers, and on the Internet hosts are known by names.
- The socket API cannot help: it sits at the transport layer, and names are a service of the application layer.
- So before connecting you run one more network exchange — a DNS query — and use its answer as the address.
- Your client is therefore a client twice: of the resolver first, of the server you actually wanted second.

The query itself is the protocol we have already looked at; what is new is the call that performs it for you.

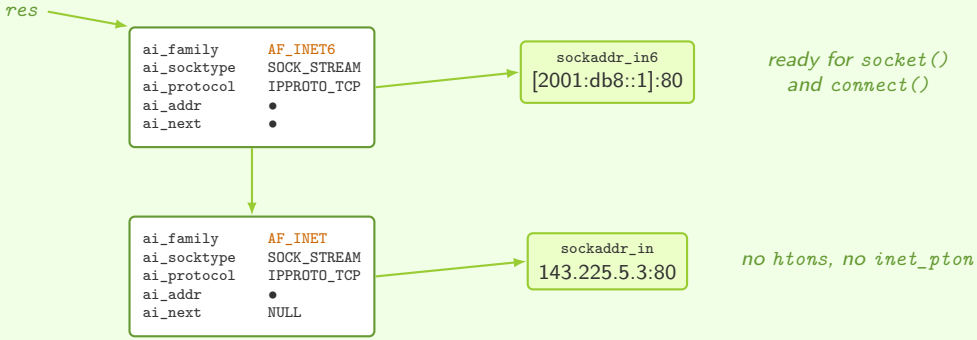
getaddrinfo: from a name to ready-made addresses

```c
struct addrinfo hints, *res;
memset(&hints, 0, sizeof hints);
hints.ai_family   = AF_UNSPEC;        /* IPv4 or IPv6, whichever exists */
hints.ai_socktype = SOCK_STREAM;      /* TCP: no datagram answers       */

int rc = getaddrinfo("www.unina.it", "80", &hints, &res);
if (rc != 0) {
    fprintf(stderr, "%s\n", gai_strerror(rc)); exit(EXIT_FAILURE);
}
```

- It resolves the name and the service together: `"80"` could be written `"http"`.
- It returns 0 or an error code of its own, printed with `gai_strerror` — not `errno`, not `perror`.
- For a server, pass `NULL` as the name with `hints.ai_flags = AI_PASSIVE` and you get the wildcard.
- Older code calls `gethostbyname`: IPv4 only, not reentrant, out of the standard. Recognise it, do not write it.

What getaddrinfo hands back



A list, not one address: one node per way of reaching that name. The first three fields are the three arguments of `socket`, and `ai_addr` is ready for `connect` as it is.

Trying the list until one works

```
int fd = -1;
for (struct addrinfo *p = res; p != NULL; p = p->ai_next) {
    fd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (fd < 0) continue;                          /* family unavailable  */
    if (connect(fd, p->ai_addr, p->ai_addrlen) == 0) break;   /* done  */
    close(fd);
    fd = -1;
}
freeaddrinfo(res);                                 /* the list is malloc'd */
if (fd < 0) { fprintf(stderr, "cannot connect\n"); exit(EXIT_FAILURE); }
```

Write this loop once and keep it: it is the same in every client, and it is what makes a program work over both IPv4 and IPv6 without knowing which one it got.

One name, several addresses

Your client works for a week and then stops, without a single line changing. What did?

The name resolved to a different address. A busy site answers with several, in an order that changes at every query, and the one you cached is now a machine that is being replaced.

- This is the load balancing we saw when a single query for a large site came back with a whole list.
- So resolve when you connect, not once at start-up, and never assume the first entry will answer.
- The loop of the previous frame is exactly that discipline written down: try, fail, take the next.

The same lookups from the shell

```
$ getent hosts www.unina.it          what getaddrinfo would return
143.225.5.3     www.unina.it

$ dig +short www.unina.it            asks the DNS directly
143.225.5.3

$ ip addr show                       which addresses this host has
```

- `getent` goes through the same resolver your program uses, so it also sees `/etc/hosts`.
- `dig` talks to the name server, which is why the two can disagree — and that difference is a bug you will meet.
- The older name for `ip` is `ifconfig`: you will find it in books, and most systems no longer install it.

Speaking HTTP by hand

```
$ nc -C www.unina.it 80
HEAD /chi-siamo/cenni-storici HTTP/1.1      typed by you
Host: www.unina.it
Connection: close
                                            blank line:  request over
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Connection: close
```

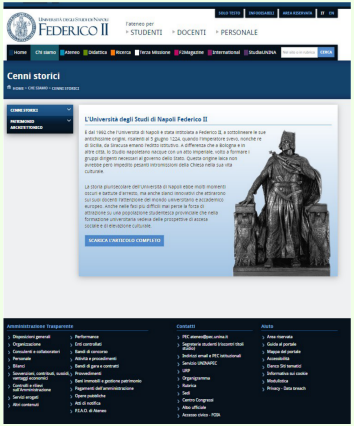
- `nc` opens a socket and copies the terminal into it: a generic client for any text protocol.
- The server cannot tell it apart from a browser, because on the wire there is nothing else to tell.
- `telnet host port` does the same and is the version you will find in older material.

The same request, written in C

We now write that by hand in a program.
A request needs exactly three things,
and you have just typed all three:

- the host name, which becomes an address;
- the port, 80 for a web server;
- the path of the object wanted.

Nothing else: the browser adds convenience, not protocol.



Resolve, then connect

```c
struct addrinfo hints, *res;
memset(&hints, 0, sizeof hints);
hints.ai_family   = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;

int rc = getaddrinfo("www.unina.it", "http", &hints, &res);
if (rc != 0) { fprintf(stderr, "%s\n", gai_strerror(rc)); exit(EXIT_FAILURE); }

int fd = connect_any(res);      /* the loop of a few frames ago */
freeaddrinfo(res);
if (fd < 0) { fprintf(stderr, "cannot connect\n"); exit(EXIT_FAILURE); }
```

No address is copied by hand and no byte is converted: the whole of the first half of this lecture is now inside `getaddrinfo`.

Send the request, read the reply

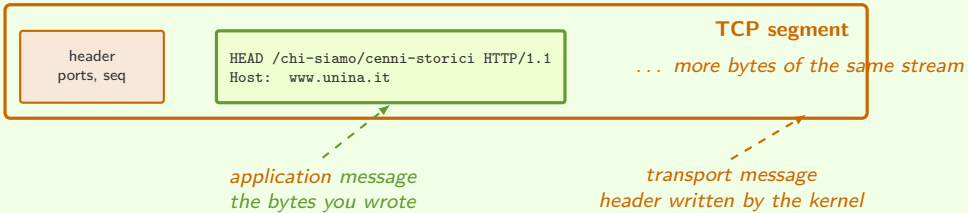
```c
const char *req = "HEAD /chi-siamo/cenni-storici HTTP/1.1\r\n"
                  "Host: www.unina.it\r\n"
                  "Connection: close\r\n\r\n";   /* the blank line ends it */

if (write(fd, req, strlen(req)) < 0) { perror("write"); exit(EXIT_FAILURE); }

char buf[4096];
ssize_t n;
while ((n = read(fd, buf, sizeof buf)) > 0)   /* until the server closes */
    fwrite(buf, 1, (size_t) n, stdout);
close(fd);
```

`HEAD` asks for the headers only. Every line ends with `\r\n`, and the blank line is what tells the server the request is complete.

What you just wrote is a payload



the transport layer moves those bytes without reading them: the payload is transparent to it

You built an application message with `strlen` and `write`, and the transport layer carried it without looking inside. Writing the request by hand is what makes that word concrete.

One read is not the reply

Drop the loop and print what a single `read` returns. What do you get?

Some prefix of the answer, of a length you cannot predict: whatever had arrived by then. The reply is a stream, and its end is not an event your program is told about.

- The loop above stops on 0, and only because `Connection: close` makes the server close when done.
- Without that header the connection stays open, the loop never ends, and the program hangs after printing everything.
- HTTP therefore says how long the body is: `Content-Length`, or chunks that announce their own size.
- Every protocol has to answer this question, and yours will have to as well.

Summary

- An endpoint is an address and a port; a connection is four numbers, and that is how two clients stay apart.
- Ports below 1024 are not yours; the kernel picks the client's out of an ephemeral range.
- Numbers that go into headers go in network byte order: `htons` for ports, `htonl` for addresses, and for your data too.
- `bind` says where I am, `connect` says where you are, and `INADDR_ANY` means every interface of this host.
- `getaddrinfo` turns a name and a service into a list of ready addresses, IPv4 and IPv6 alike.
- An application message travels as the payload of a transport message, and nothing below reads it.

Your chat, told where to connect

What to change

The chat you wrote has `127.0.0.1` and `5000` in its source. Make the client take a host and a port from `argv` and resolve the host with `getaddrinfo`; make the server take its port the same way.

- The server keeps `INADDR_ANY`: it must answer on the interface the other machine can reach.
- Test: run the server on one machine and the client on another, by name if there is one, by address otherwise.
- Nothing else changes — and from here on every version of the project will be started this way.

With the two machines connected, what do the four numbers in `ss -tn` look like on each side?